

960121

BACKEND & REST APIS

Session 05



OBJECTIVE

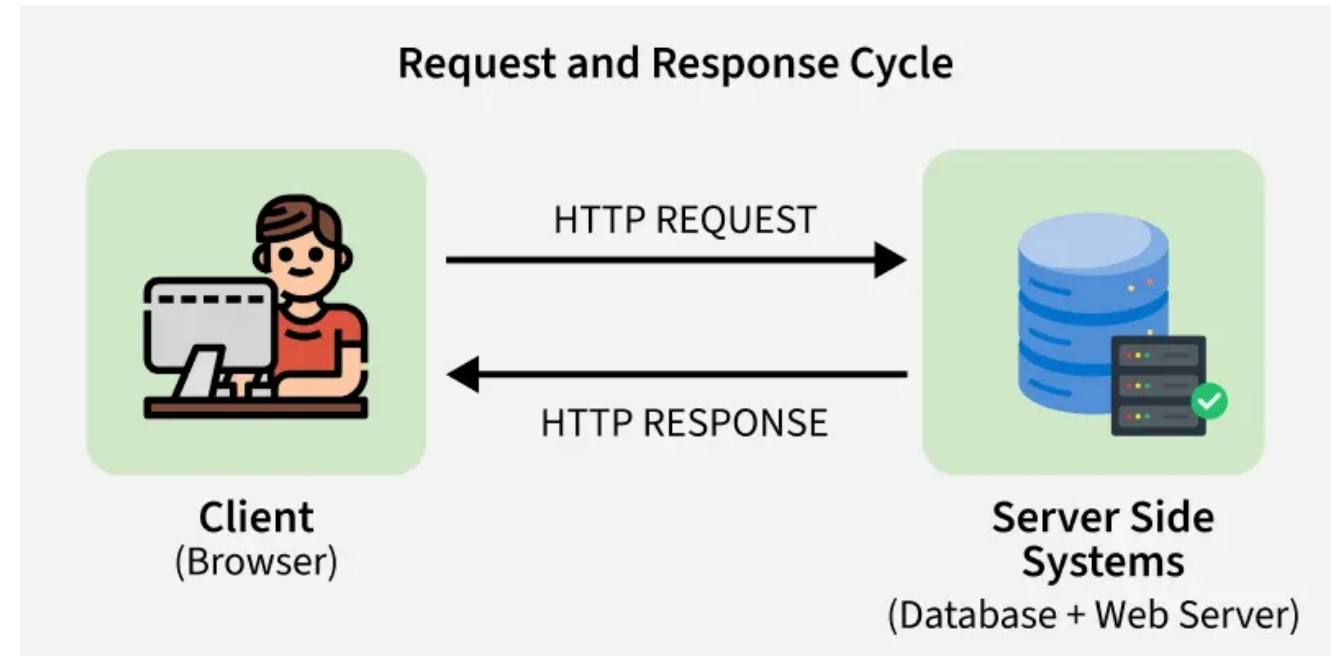
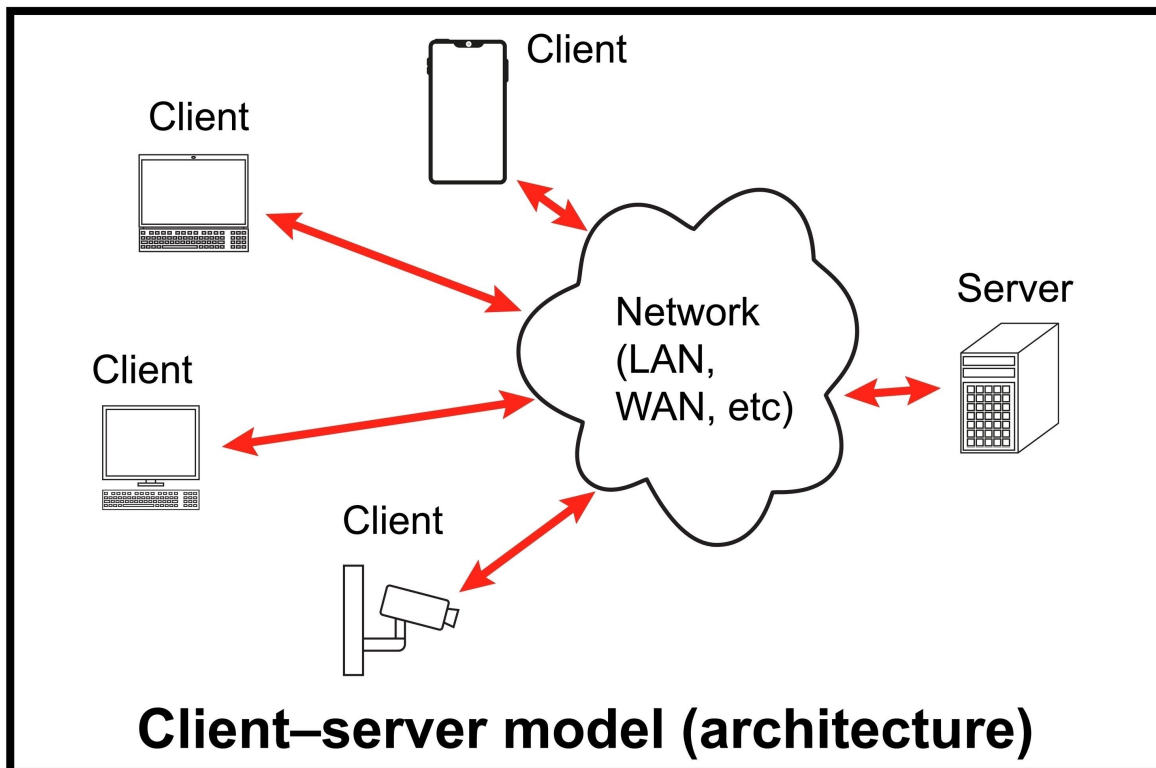
- In this session, students stop thinking about "files" and start thinking about **Services**.
- We introduce **Modular Thinking**: the idea that the server is a black box that provides data upon request, regardless of who is asking.
- On the frontend, the code is **resident** (living in the user's browser).
- On the backend, the code is a **resident of the cloud**, serving thousands of people simultaneously.

THE SERVER-SIDE MENTAL MODEL

To transition to the backend, you need to know the **architecture of communication**:

- **Node.js & Express (The Environment):** Understanding that JavaScript can run outside the browser to manage files and network requests.
- **RESTful Principles:** Learning the "grammar" of the web.
 - **GET** (Read), **POST** (Create), **PUT** (Update), **DELETE** (Destroy).
- **Modular Architecture:** Separating the **Routes** (the URLs) from the **Controllers** (the logic). This prevents the "God File" problem where one file does everything, like a PHP script.

CLIENT-SERVER MODEL



1. THE 'REQUEST-RESPONSE' LIFE CYCLE

Think of a server as a **Message Processor**.

- **The Mental Model:** The server is like a waiter in a restaurant. It doesn't give you the whole kitchen; it takes your specific "order" (**Request**), goes to the kitchen (**Logic/DB**), and brings back only what you asked for (**Response**).
- **The Technical Reality:** Every interaction (Request-Response message) has a **Header** (metadata like "Who am I?" and "What format do I want?") and a **Body** (the actual data).

REQUEST-RESPONSE LIFECYCLE

1. **Initiation & DNS Lookup:** The user initiates a request (e.g., entering a URL or clicking a link). The browser uses the DNS server to translate the domain name into an IP address to locate the server.
2. **TCP/TLS Handshake:** A secure connection is established between the client and server (TCP 3-way handshake + TLS negotiation), usually taking ~100ms.
3. **Sending the Request:** The client sends the HTTP request, which includes a method (GET, POST), URL, headers, and optional data.

REQUEST-RESPONSE LIFECYCLE

4. **Server Processing:** The server receives the request, passes it through middleware for authentication/logging, and processes it via the application layer.
5. **Generating the Response:** The server creates a response object, which includes a status code (e.g., 200 OK or 404), headers, and the requested content.
6. **Sending the Response:** The server sends the processed response back to the client over the established connection.
7. **Rendering the Content:** The browser receives the response, parses the HTML/CSS/JS, and renders the page for the user.

RESTFUL

- A **RESTful** API (Representational State Transfer) is a standardized architectural style for web services that enables secure, efficient, and stateless communication between systems via HTTP/HTTPS.
- It uses standard HTTP methods (GET, POST, PUT, DELETE) to manipulate data, typically in JSON format, facilitating interoperability between diverse applications.

KEY CONCEPTS OF RESTFUL API

- **Resources & URIs:** Data is organized as resources (e.g., users, products) accessed via unique URIs (e.g., /users/1).
- **HTTP Methods:**
 - **GET:** Retrieve data.
 - **POST:** Create new data.
 - **PUT/PATCH:** Update existing data.
 - **DELETE:** Remove data.
- **Statelessness:** Each request contains all necessary information; the server does not store client session data, improving scalability.
- **JSON/XML:** Data is typically exchanged in **lightweight formats**.

2. STATELESSNESS

On the frontend, variables stay in memory as long as the page is open. On the backend, the server "forgets" everything the moment it sends a response.

- **The Mental Model:** Imagine every time you speak to a friend, they have total amnesia. You have to re-introduce yourself every single time.
- **The Logic:** This is why we send **Tokens** or **IDs** with every request. If you want to "GET /profile," the server doesn't know whose profile unless the request specifically says `GET /profile?user=123`.

3. THE "GATEKEEPER" PATTERN

For **Security & Validation**

On the frontend, "validation" is for User Experience (e.g., showing a red box if a password is too short). On the backend, "validation" is for **Survival**.

- **The Mental Model:** The frontend is the "Storefront"—it looks nice and guides the user. The backend is the "Bank Vault"—it doesn't care how nice the user is; it only cares if the key is correct and the math adds up.

3. THE "GATEKEEPER" PATTERN

- **Critical Thinking:**

- If a user changes a product's price to \$0 in the Browser Console and clicks 'Buy,' who is responsible for catching that?

“The Server-Side Mental Model assumes the client is untrustworthy.”

3. THE "GATEKEEPER" PATTERN

In a Full-Stack architecture, the Frontend and Backend have completely different responsibilities regarding data.

- **The Mental Model: "Trust but Verify"**
 - **Frontend (The Concierge):** Helps the "good" user by showing errors early (e.g., "This field is required"). It aims for Speed and UX.
 - **Backend (The Gatekeeper):** Assumes every request could be a "bad" actor or a mistake. It aims for **Integrity and Security**.

THE 3 LAYERS OF THE GATEKEEPER

Layer	Technical Action	Business Logic Reason
1. Authentication	"Who are you?"	Ensure only registered users can place orders.
2. Authorization	"Are you allowed to do this?"	A customer shouldn't be able to DELETE a product from the catalog.
3. Validation	"Is this data sensible?"	Preventing a user from ordering -5 t-shirts or a \$0 laptop.

4. ENVIRONMENT VS. EXECUTION

You need to understand that `window` and `document` do not exist on the server.

Feature	Browser (Frontend)	Node.js (Backend)
Global Object	<code>window</code>	<code>global</code>
Primary Task	UI Rendering / Interaction	Data Processing / File System
Security	Open (User can see all code)	Closed (User only sees the API)
Performance	Limited by user's device	Scalable by adding server power



REFERENCE

- **What is REST?** (<https://restfulapi.net/>)

CLASS ACTIVITY:

ARCHITECTING THE API

1. The Scenario:

- **The Challenge:** Currently, your product list is a static JSON file on the frontend. If we change a price, every user has to download a new app version.
- **Goal:** We will move the product data to an Express server. The frontend will "ask" the server for the product list via a REST API.

ARCHITECTING THE API

To visualize the communication between the Frontend and the Backend, we use a **Request/Response Map**. This is essentially a "Contract" between the two sides of the application.

Request/Response Map: The Product Catalog

This map illustrates how a single user action (clicking a category) triggers a cycle of data exchange.

ARCHITECTING THE API

The Logical Flow

1. **Trigger:** The browser loads the webpage.
2. **Request:** The Browser sends a specific "envelope" to the Server.
3. **Processing:** The Server opens the envelope, checks the "Gatekeeper" logic, and fetches data.
4. **Response:** The Server sends back a "Package" (JSON) and a "Status" (Success/Fail).

ARCHITECTING THE API

The "Contract" Table: how the “envelope” looks like.

Component	Request (The Order)	Response (The Delivery)
Method	GET	—
Endpoint (URL)	/api/products	—
Headers	Accept: application/json	Content-Type: application/json
Status Code	—	200 OK (or 404 Not Found)
Body (Data)	<i>(Empty for GET)</i>	[{"id": 1, "name": "Laptop", "price": 999}, ...]

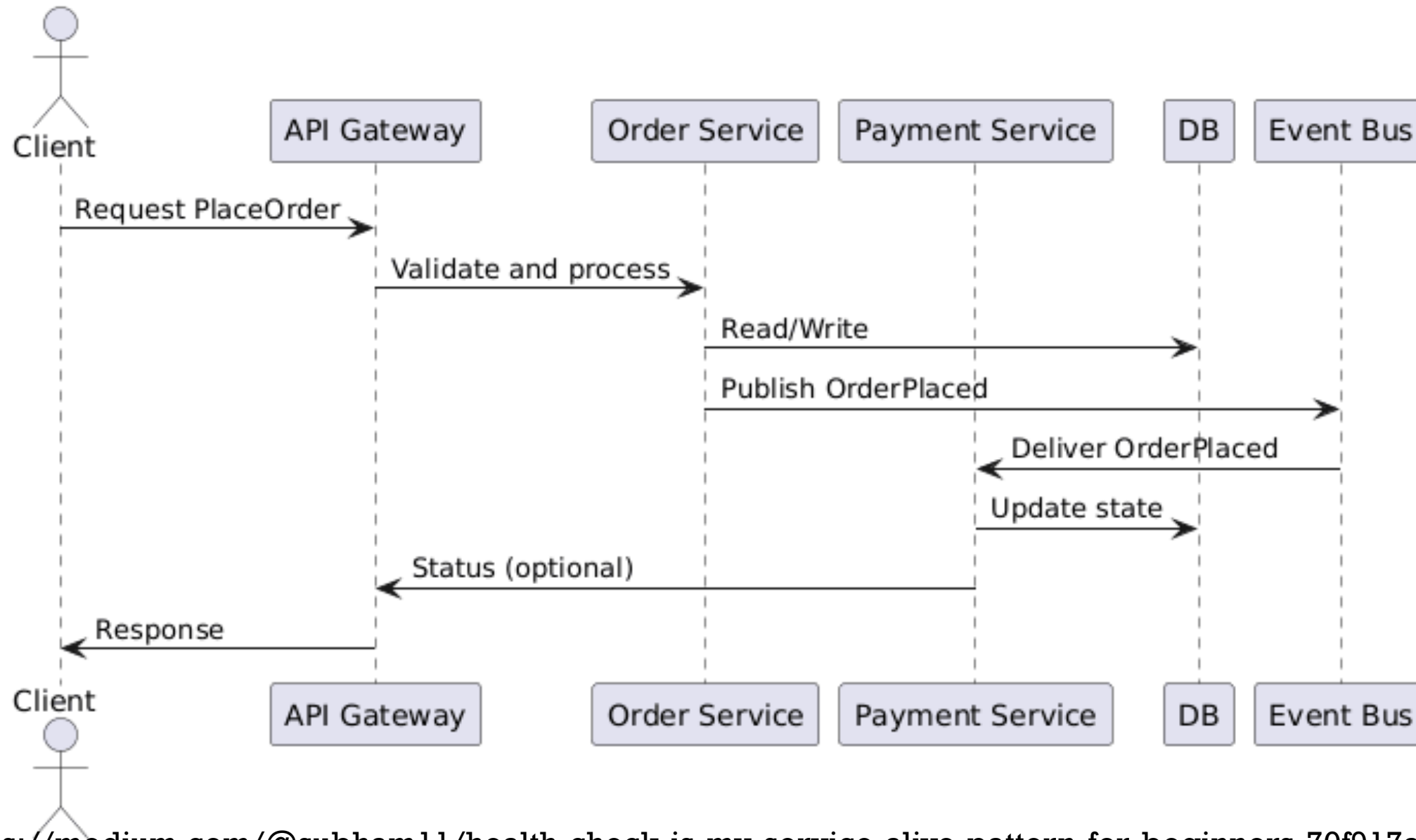
ARCHITECTING THE API

2. (15 min.) UML:

- Draw a **Sequence Diagram** for Request/Response Map.
- **Logic Check:** You must map the URL to the action.
 - **Example:** `GET /api/products` → returns all items.
 - **Example:** `GET /api/products/:id` → returns one specific item.

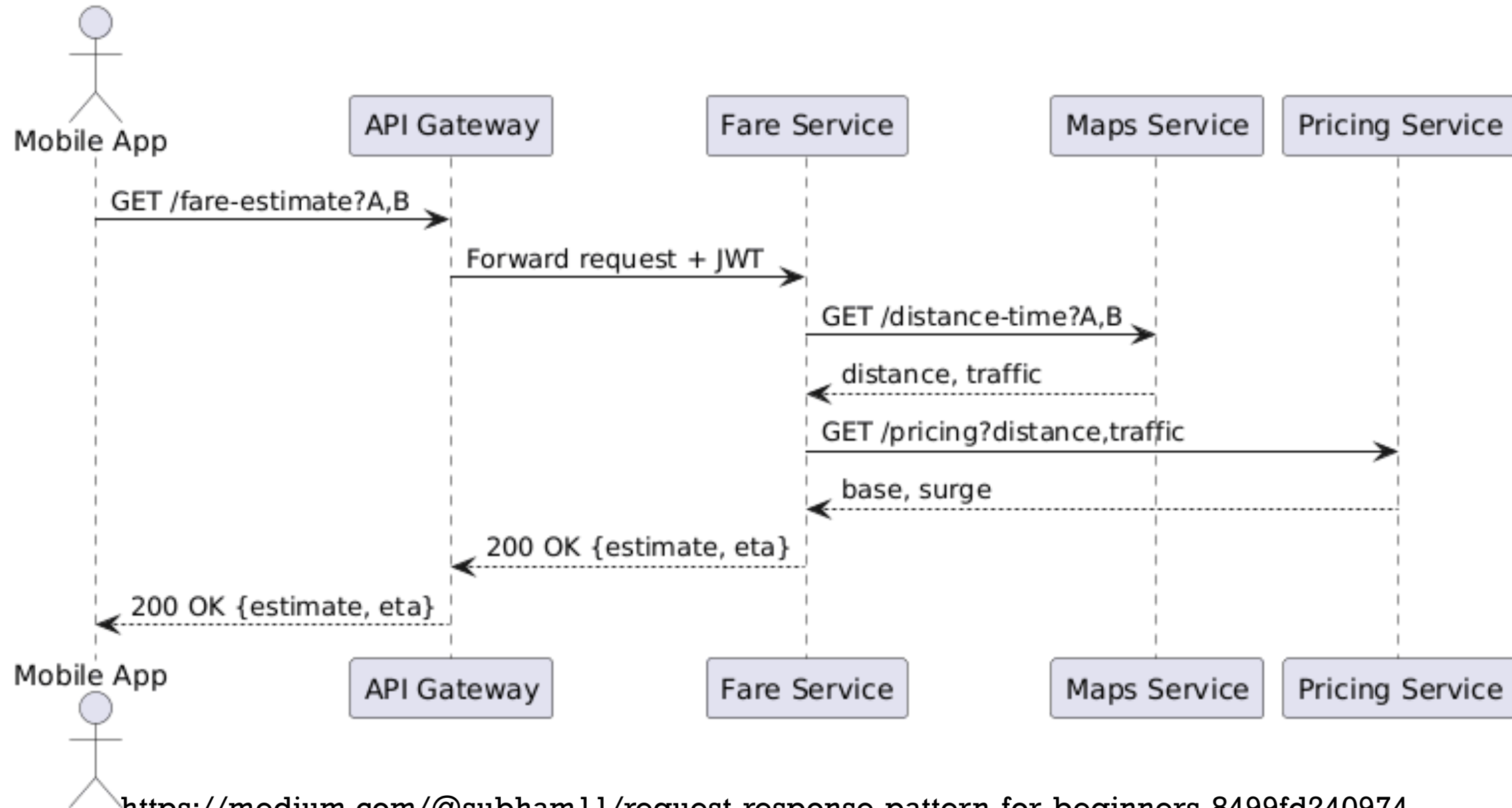
EXAMPLE

Health Check — Core Interaction



EXAMPLE

Request/Response — Fare Estimate (Ride-hailing)



ARCHITECTING THE API

3. (5 min.) **Peer Evaluation:**

- Students swap drawings.
- **Critical Thinking Question:**
 - Does your friend's API design follow REST naming conventions? (e.g., is it `/getProducts` [Action-based] or `/products` [Resource-based]?)
 - Explain why resources are better for scaling.
 - Ask your friend to add these "**What if the Server is down?** (Response: `500 Internal Server Error`)" scenarios to their hand-drawn map.

ARCHITECTING THE API

4. (5 min.) GenAI Prompt Engineering :

Prompt: (prepare Node.js and Express)

“I am using macOS. Help me install Node.js and Express on my computer. I need a step-by-step guideline for installing and testing this backend environment.”

ARCHITECTING THE API

4. (5 min.) GenAI Prompt Engineering:

- Students use UML logic to craft a structured prompt.

Prompt:

“Act as a Backend Architect. Using Node.js and Express, create a modular folder structure. Write a `products.js` route file that handles a GET request to `/api/products`. The data is currently stored in a local JSON file. Ensure the code follows the Controller-Route-Service pattern.”

ARCHITECTING THE API

5. (20 min.) **Integration:**

- Open your project in VS Code.
- Initialize your backend using `npm init`.
- Integrate the generated code and run your first local server using the VS Code terminal.

ARCHITECTING THE API

6. (20 min.) **Debugging & Testing:**

- **Tool Discovery:** Students use the browser or a tool like Postman/Thunder Client to "hit" their API directly without the frontend.
- **The "CORS" Moment:** You will try to connect your Session 4 frontend to this new backend, and it will fail due to CORS (Cross-Origin Resource Sharing).
- **The Lesson:** This is a security feature, not a bug. You must learn to configure the backend to "trust" your frontend.

ARCHITECTING THE API

7. (5 min.) Version Control:

- Terminal Command:

- `git add`

- `git commit -m "feat: setup express server and initial RESTful product routes."`

- `git push.`

- VS Code:

- GitHub Desktop:

KEY TAKEAWAY

- **The Client is Replaceable:** If we build the API correctly, we could build a Mobile App tomorrow that uses the exact same backend logic.
- **Statelessness:** The server doesn't "know" who the user is between requests unless we tell it. Each request must be self-contained.
- **Safety First:** The backend is the "Gatekeeper." We never trust the frontend. (e.g., the frontend says a product costs \$1, but the backend checks the "True" price in the database).

WEEKEND WORK (5 POINTS)

Modify your code to follow this **Logic Flow**:

1. **Trigger**: User clicks "Hat" on the Frontend.
2. **Request**: The Browser sends a specific "envelope" to the Server.
3. **Processing**: The Server opens the envelope, checks the "Gatekeeper" logic, and fetches data.
4. **Response**: The Server sends back a "Package" (JSON) and a "Status" (Success/Fail).

WEEKEND WORK (5 POINTS)

1. Write the "Contract" Table.
2. Write a Sequence Diagram for Request/Response Map.
3. Write GenAI Prompt from the "Contract" table + Sequence Diagram
 - `Express.js` route for this request.
 - Accept a **query parameter** for 'category.'

Ensure your code has comments explaining how it works!!

WEEKEND WORK (5 POINTS)

Submission:

1. Digital version (.pdf or image) of your:
 - Contract table
 - Sequence Diagram
 - GenAI Prompt(s)

Placed them in `Documentation/product_filter`

2. `git commit -m "feat: weekend work"`
3. GitHub commit before Sunday 11.59 PM